Fake Vision



By:

Sania Batool

23756

Ayesha

24764

Sana Anwar

30486

Supervised by:

Dr. Tahira Nazir

Faculty of Computing

Riphah International University, Islamabad

Fall 2020-2024

A Dissertation Submitted To

Faculty of Computing,

Riphah International University, Islamabad

As a Partial Fulfillment of the Requirement for the Award of the Degree of

Bachelors of Science in Computer Science

Faculty of Computing

Riphah International University, Islamabad

Date: 11th June, 2024

Final Approval

This is to certify that we have read the report submitted by ***Sania Batool (23756), Ayesha (24765), and Sana Anwar (30486)***, for the partial fulfillment of the requirements for the degree of the Bachelor of Science in Computer Science (BSCS). It is our judgment that this report is of sufficient standard to warrant its acceptance by Riphah International University, Islamabad for the degree of Bachelors of Science in Computer Science (BSCS).

Committee:

1

_____

Dr. Tahira Nazir
(Supervisor)

2

_____

Dr. Tahira Nazir
(Head of Department)

Declaration

We hereby declare that this document "**Fake Vision**" neither as a whole nor as a part has been copied out from any source. It is further declared that we have done this project with the accompanying report entirely based on our efforts, under the proficient guidance of our teachers, especially our supervisor **Dr. Tahira Nazir**. If any part of the system is proved to be copied out from any source or found to be a reproduction of any project from anywhere else, we shall stand by the consequences.

_____

Sania Batool

23756

_____

Ayesha

24764

_____

Sana Anwar

30486

Dedication

First, we are obliged to Allah Almighty the Merciful, the Beneficent, and the source of knowledge, for granting us the courage and knowledge to complete this Project. We dedicate our work to our respected parents and motivated supervisor "**Dr. Tahira Nazir**" who has been our constant source of inspiration and support which led us to the success of this project. She motivated us and guided us throughout the whole journey from the selection of the topic to the completion of this project.

Acknowledgement

First, we are obliged to Allah Almighty the Merciful, the Beneficent, and the source of knowledge, for granting us the courage and knowledge to complete this Project. We dedicate our work to our respected parents and motivated supervisor "**Dr. Tahira Nazir**" who has been our constant source of inspiration and support which led us to the success of this project. She motivated us and guided us throughout the whole journey from the selection of the topic to the completion of this project.

Sania Batool

23756

Ayesha

24764

Sana Anwar

30486

Abstract

Deep learning algorithms have become so powerful due to increasing processing power that it is now quite easy to produce an identical human-synthesized video, sometimes known as a "deep fake." It is easy to imagine scenarios in which these life-like face-swapping deep fakes are used to incite political unrest, stage terrorist acts, produce revenge porn, or blackmail individuals. In this study, we present a novel deep learning approach that successfully differentiates between artificial intelligence (AI)-generated phony movies and actual ones. Our technique can recognize replacement and recreation deep fakes automatically. Our goal is to combat artificial intelligence (AI) by using it against it.

Our system extracts frame-level features using a Res-NeXt Convolution neural network and then uses these features to train a Recurrent Neural Network (RNN) based on Long Short-Term Memory (LSTM) to classify videos based on whether or not they have been altered, i.e., whether they are deepfake or authentic. We assess our technique on a significant quantity of balanced and mixed data produced by mixing the different accessible data, such as Face-Forensic++, Deepfake detection challenge, and Celeb-DF, to imitate the real-time scenarios and improve the model's performance on real-time data. We also demonstrate how our technique may produce competitive results using an incredibly straightforward and reliable method.

Keywords:

Deepfake, Extraction Res-NeXt Convolution neural network, Recurrent Neural Network (RNN), Long Short-Term Memory (LSTM).

Table of Contents

Table of Contents i
List of Tables ii
List of Figures iii
Abstract 1
Chapter 1: Introduction 2
1.1 Goals and Objectives 2
1.2 Scope of the Project 2
Chapter 2: Literature Review 4
2.1 Introduction 4
2.2 Background and Problem Elaboration 4
2.3 Detailed Literature Review 4
2.3.1 Definitions 5
2.3.2 Related Research Work 1 6
2.4 Literature Review Summary Table 7
2.5 Research Gap 7
2.6 Problem Statement 7
Chapter 3: Requirements and Design 9
3.1 Requirements 9
3.1.1 Functional Requirements 9
3.1.2 Non-Functional Requirements 10
3.1.3 Hardware and Software Requirement 10
3.2 Proposed Methodology 11
3.3 System Architecture 12
3.4 Use Cases 13
3.4.1 Login 14
3.4.2 Sign Up 15
3.4.3 Detection 16
3.4.4 Community Forum 17
3.4.5 Logout 18
3.5 GUI Graphical User Interfaces 20
Chapter 4: Implementation and Test Cases 28
4.1 Implementation 28
4.1.1 Implementation of First Component/Algorithm 29
4.2 Test case Design and description 30
4.2.1 Sample Test Case No.1 30
4.2.2 Sample Test Case No.3 31
4.2.3 Sample Test Case No.4 32
4.2.4 Sample Test Case No.5 33
4.2.5 Sample Test Case No.6 33
4.2.6 Sample Test Case No.7 34
4.2.7 Sample Test Case No.8 35
4.2.8 Sample Test Case No.9 36
4.2.9 Sample Test Case No.10 37
4.3 Test Metrics 37
4.3.1 Sample Test case Matric.No.1 37
Chapter 5: Experimental Results and Analysis 39
Chapter 6: Conclusion and Future Directions 42
References 44

List of Tables

Table 1: Literature Review 5
Table 2: History of Deepfake Video Detection Tools 7
Table 3: Functional Requirements 9
Table 4: Login User 14
Table 5: Sign Up 15
Table 6: Detection 16
Table 7: Community Forum 17
Table 8: Logout 18
Table 9: Test Case 1 30
Table 10:Test Case 2 30
Table 11: Test Case 3 31
Table 12: Test Case 4 32
Table 13: Test Case 5 33
Table 14: Test Case 6 34
Table 15: Test Case 7 34
Table 16: Test Case 8 35
Table 17: Test Case 9 36
Table 18: Test Case 10 37
Table 19: Test Metrices 37

List of Figures

Figure 1: System Architecture 12
Figure 2: Use Case 13
Figure 3: Splash Screen 20
Figure 4: Choose Media 21
Figure 5: Upload Video/Image 22
Figure 6: Results 23
Figure 7: About 24
Figure 8: Community Forum 25
Figure 9: User Profile 26
Figure 10: History 27
Figure 11: Accuracy Curve 40
Figure 12: Loss Curve 41
Figure 13: Confusion Matrix 41

Abstract

Deep learning algorithms have become so powerful due to increasing processing power that it is now quite easy to produce an identical human-synthesized video, sometimes known as a "deep fake." It is easy to imagine scenarios in which these life-like face-swapping deep fakes are used to incite political unrest, stage terrorist acts, produce revenge porn, or blackmail individuals. In this study, we present a novel deep learning approach that successfully differentiates between artificial intelligence (AI)-generated phony movies and actual ones. Our technique can recognize replacement and recreation deep fakes automatically. Our goal is to combat artificial intelligence (AI) by using it against it.

Our system extracts frame-level features using a Res-NeXt Convolution neural network and then uses these features to train a Recurrent Neural Network (RNN) [1] based on Long Short-Term Memory (LSTM) [2] to classify videos based on whether or not they have been altered, i.e., whether they are deepfake or authentic. We assess our technique on a significant quantity of balanced and mixed data produced by mixing the different accessible data, such as Face-Forensic++ [3], Deepfake detection challenge [3], and Celeb-DF [3], to imitate the real-time scenarios and improve the model's performance on real-time data. We also demonstrate how our technique may produce competitive results using an incredibly straightforward and reliable method.

Keywords:

Deepfake, Extraction ResNeXt Convolution neural network, Recurrent Neural Network (RNN), Long Short-Term Memory (LSTM).

Chapter 1: Introduction

In the modern world, where technology can change what we see and hear, deepfakes have become increasingly prominent. Deepfakes are a type of synthetic media that uses artificial intelligence (AI) to fabricate videos, images, or audio recordings. This manipulated content can spread wrong information, putting a big risk on trust and truth. So, we present our app **"Fake Vision"** which uses cutting-edge technology to discern real content from manipulated content.

1.1 Goals and Objectives

The main goal of our app "Fake Vision" is to accurately detect deepfakes and ensure the integrity of digital media. Our app will help people to identify false information. The app will also have a community forum. It is a platform for users to discuss deepfakes and other related topics. Users will be able to share their knowledge, collaborate on projects, and learn from others as well.

1.2 Scope of the Project

As our lives become more digital, deepfakes are becoming a common problem that affects many parts of it including social media, law enforcement, news, and politics. Through "Fake Vision" people can identify the difference between real and fake.

1.2.1 Social Media:

Social media has a lot of fake content, lies, and hate spread throughout. Our deepfake detection app "Fake Vision" helps to keep people safe and make the online world more trustworthy.

1.2.2 Law Enforcement:

In the realm of law enforcement, deepfakes can be wickedly employed to create false evidence and wrongfully frame innocent individuals. Law enforcement agencies can use our app to enhance their investigative capabilities, ensuring justice and protecting the public.

1.2.3 News and Journalism:

Deepfakes make it hard to trust news and journalism because they make people question that if the content they are reading or watching is real. Our app would make it easy to check if the news is real, making sure that people have reliable information.

1.2.4 Politics:

In politics, deepfakes can manipulate people and mess up elections, which isn’t good for democracy. Our app can help safeguard the integrity of the elections and make sure that voters have the right facts to make informed choices.

Chapter 2: Literature Review

2.1 Introduction

With the advancements in the digital era and technologies, the generation and emergence of deepfake video also impact people by making them conscious about their privacy and integrity of information. Deepfake Videos generated by advanced machine learning algorithms generate hyper-realistic deepfake videos that a common man cannot differentiate whether the video he is watching is real or not.

With these advancements in digital media, it has become a pressing concern for the stakeholders to develop tools and software that help mitigate and detect deepfake videos. Deepfake video detection tools not only preserve the integrity of data in law enforcement and journalism but it is also important to have the public trust in digital media.

This literature review attempts to provide a comprehensive detail of the existing deepfake video detection tools. We have highlighted the research, methodologies, and algorithms that are used in this domain and provide future advancements in the technology and algorithms by presenting our deepfake video detection App **Fake Vision.**

2.2 Background and Problem Elaboration

The generation and proliferation of deepfake videos have increased in recent years. The trend that was started in 2016 with the emergence of Face2Face papers has reshaped itself and there are now thousands of deepfake video detection apps and tools. With the advancement of the generation of deepfake video technology, the common man can't distinguish between real and fake videos. Consequently, people have lost their interest in the digital media.

There is no publicly available free mobile application that can detect deepfake videos. Forensic experts charge people a lot of money to tell them the integrity of the data. Deepfake videos have created a massive impact in the world of politics and journalism. The deepfake video of Putin, Obama and the recently released deepfake video of Ukrainian Prime spreads like fire and it also. Thus, it has become a pressing concern for the stakeholders to develop tools and software that help mitigate and detect deepfake videos.

2.3 Detailed Literature Review

This Mobile Application will help users to detect their deepfake videos and Images. We have extended our research in this domain by adding advanced machine learning algorithms LSTM

and RNN. The algorithms will help in detecting fake videos and Images made with face swapping. Here is the detailed Literature review of our Fake Vision App.

Detection:

Detecting deepfake videos and Images remains a significant challenge, as there isn't a widely-used app that can reliably identify them. Furthermore, the one existing app, Deepware [6] is not compatible with newer Android versions. This underscores the importance of further research and development in the field of deepfake detection.

Media Integrity Score:

In our Fake Vision app, we're introducing an innovative feature known as the Media Integrity Score. This score provides users with a practical way to assess how authentic or fake a given video or image is, adding an important dimension to our app's functionality in combating misinformation and deepfake content. This feature is only available in the "Deepware" Web App.

Community forum:

In our Fake Vision app, we've introduced a community forum to encourage transparency and collaboration within the evolving landscape of deepfake technology. This forum provides users with a space to engage in discussions, seek advice, and share knowledge, reinforcing our dedication to working together to combat digital deception.

Table 1: Literature Review

Features	Deepware	Fake Vision
Video Detection	✓	✓
Image Detection	X	✓
Community Forum	X	✓
Confidence of Prediction	✓	✓
FaceSwap Detection	✓	✓

2.3.1 Definitions

Deep Learning:

Deep learning is a branch of Machine learning, which is a subset of Artificial Intelligence. In the deep learning technique, we train large amounts of data on the neural network then we extract features from it. We also extract patterns from this analysis. Deep learning is widely used in image recognition, Face Recognition, Natural Language Processing, Computer vision, and many other fields.

Deepfakes:

Deepfake is the manipulated media that is synthesized by using deep learning techniques. By using deep learning techniques, the real videos, images, or audio are altered to create fake convincing content. Deepfakes are created to impersonate someone by creating fabricated content.

Long Short-Term Memory (LSTM):

A particular kind of recurrent neural network (RNN) [1] architecture called Long Short-Term Memory (LSTM) [2] was created to solve the vanishing gradient issue that frequently arises during regular RNN training. Text, audio, video, and time series data are examples of sequential data that LSTMs are particularly good at processing and predicting.

The fundamental components of an LSTM unit are an input gate, an output gate, and a forget gate. The cell state acts as a memory to retain information across extended sequences. Based on the input data and the internal state of the network, these gates control the flow of information into and out of the cell state, enabling LSTMs to selectively remember or forget information.

RNN:

Recurrent Neural Network is an artificial Intelligence neural network that takes input and output sequentially. RNN maintains a memory of previous input, which is why it is best for temporal analysis. In deepfakes, RNN is used for the detection of lip-syncing, voice recognition, and speech synthesis.

2.3.2 Related Research Work 1

Website Name: Deepware Scanner

Main Service: Detects Deepfake Videos

Details: Existing systems detect deepfake videos generated by face swapping, it also generates an integrity score to tell how much fake a video is.

2.4 Literature Review Summary Table

The columns in the table depend upon your problem and should be specific to your project.

Table 2: History of Deepfake Video Detection Tools

No.	Name, reference	Inventor	Year	Input	Output	Description
1.	Deepwater Scanner	Zemana Company	2018	Face Swapped Deepfake Video	Deepfake Video detected with the integrity score.	Zemana Company along with its deepware AI team started their research in 2018 on the detection of deepfake video made with face swapping. And they successfully built a deepfake video detection app called Deepware Scanner.

2.5 Research Gap

Deepware is the only publicly available Deepfake detection web App. The app version of Deepware is not compatible with the new version of Android mobiles. Another problem with the existing system is that it accurately detects only the deepfake videos that are made by face swap but it faces the problem in accurately detecting images that are made by using face swapping.

Our Project for Detection of Deepfake Videos will be the first ever publicly available deepfake video detection app, which will be compatible with the older and newer versions of Android. Unlike deepware scanner, users can also detect deepfake images and videos.

2.6 Problem Statement

Visual effects have been used for decades to display convincing modifications of digital photos and movies; however, new developments in deep learning have significantly increased the

realism of false material and the simplicity with which it may be produced. These so-called artificial intelligence-generated materials, are also known as "deep fakes". Using artificially intelligent technologies to create Deep Fakes is an easy process. However, identifying these Deep Fakes is a significant difficulty.

There are several historical instances when deepfakes have been employed as an effective tactic to incite political unrest, stage terrorist attacks, blackmail individuals, etc. Therefore, it becomes crucial to identify these deepfakes and stop them from spreading over social media. We have made progress in identifying deepfakes by employing an artificial neural network based on long short-term memory.

Chapter 3: Requirements and Design

Our project's main goal is to develop a reliable deepfake detection tool with precise specifications. Key user functionalities, including sign-up, login, video upload, accuracy evaluation, and community interaction, are outlined in the Functional Requirements. To enhance them, the Non-Functional Requirements emphasize an easy-to-use interface and a high level of detection accuracy. There are few hardware requirements, and the software tools that must be used include Firebase, PyTorch, Python, and Flutter. This chapter provides a fundamental manual for creating our Deepfake Detection App for Fake Vision.

3.1 Requirements

3.1.1 Functional Requirements

Functional requirements explain what the application must do. These are behavioral requirements that name the actual services that a user wants from the application. Following are the functional requirements of the application.

FR-0.1 User Requirements

Table 3: Functional Requirements

ID	Requirement
FR-1.1	Users shall be able to sign up for the application.
FR-1.2	Users shall be able to log in to the application.
FR-1.3	Users shall be able to view the home screen.
FR-1.4	Users shall be able to upload the video to be detected.
FR-1.5	Users shall be able to view the accuracy score of the video.
FR-1.6	Users shall be able to share their thoughts through a community forum.
FR-1.7	Users shall be able to log out of the application.

3.1.2 Non-Functional Requirements

Usability: The user interface of the application will be simple, easy to use, and user-friendly.

Reliability: The application will be reliable by detecting deep fake videos with high accuracy.

3.1.3 Hardware and Software Requirement

Hardware Requirements:
Our project involves GPU and fast SSDs, but due to limited resources, we are using Google Colab.

Software Requirements
Development Environment:

Operating System: Any operating system that will handle Flutter, Python, PyTorch, and Deep Learning.

IDE: Visual studio, Google Colab, Android Studio.

Version Controls: Git and GitHub.

Front End Development:

Flutter: Flutter for designing User Interface.

Dart: For adding logic to the User Interface.

Firebase: For the backend services like for CRUD operation.

Training data with ML and Integration:

Python: Python programming for the implementation of Deep Learning Algorithms such as LSTM and ResNeXt.

PyTorch: PyTorch framework for training neural networks.

Additional Libraries: Additional libraries as required for data processing.

3.2 Proposed Methodology

We examined the issue description and determined if the proposed solution is workable. after determining if the problem statement is feasible. The collection and analysis of the dataset is the next stage. We examined the data set using many training methodologies, such as positively or negatively trained, that is, training the model solely with false or actual videos. However, we discovered that this might introduce additional bias into the model, resulting in faulty predictions. Thus, after doing extensive study, we discovered that the optimal method for avoiding bias and variation in the algorithm and achieving acceptable accuracy is to train it in a balanced manner.

We examined our options and chose to program using the Python 3 language and the PyTorch framework. PyTorch is selected because it is very customizable and provides strong support for CUDA, or the Graphics Processing Unit (GPU). To train the final model on a large number of data sets, use the Google Colab Platform.

3.3 System Architecture

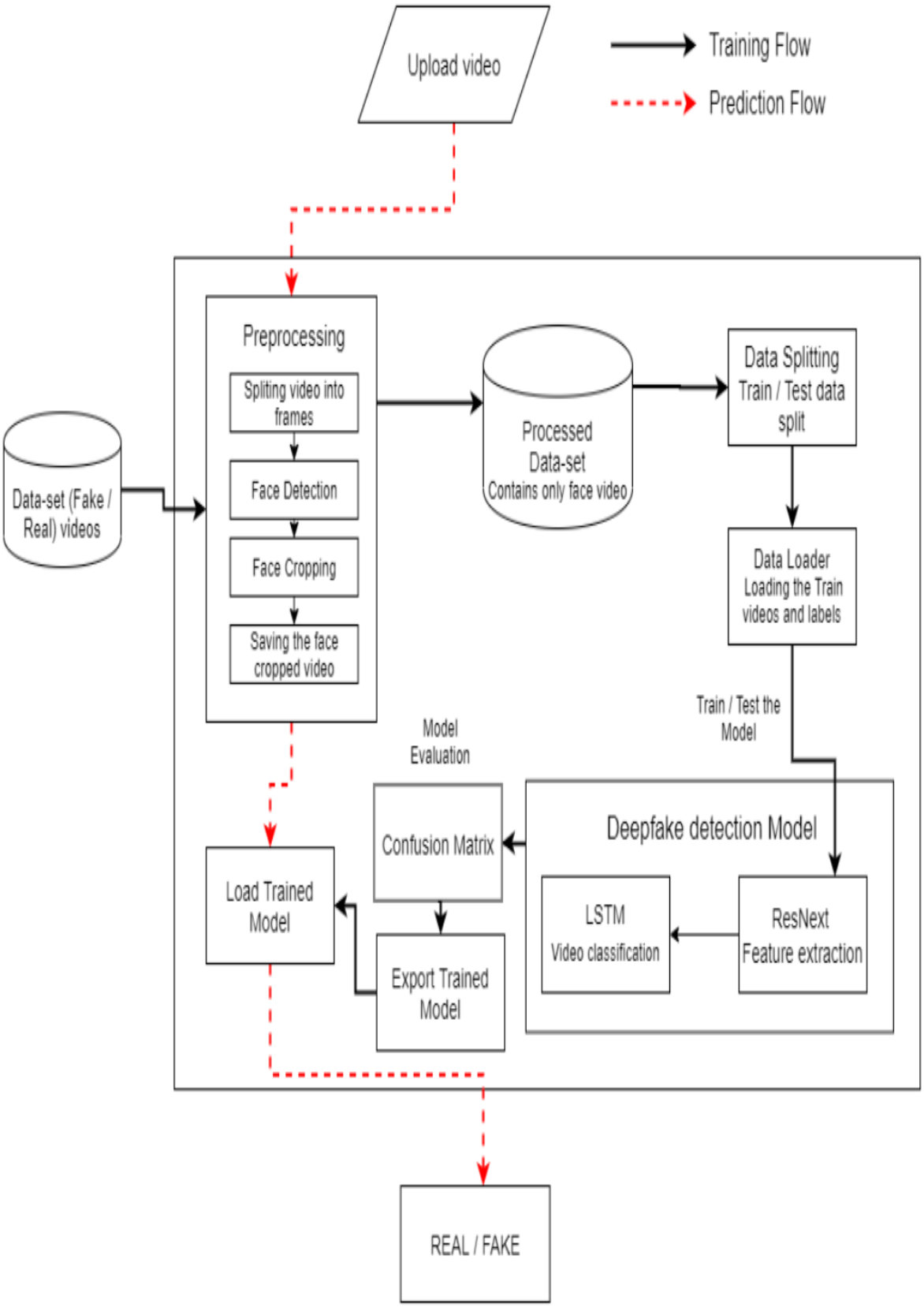


Figure 1: System Architecture

3.4 Use Cases

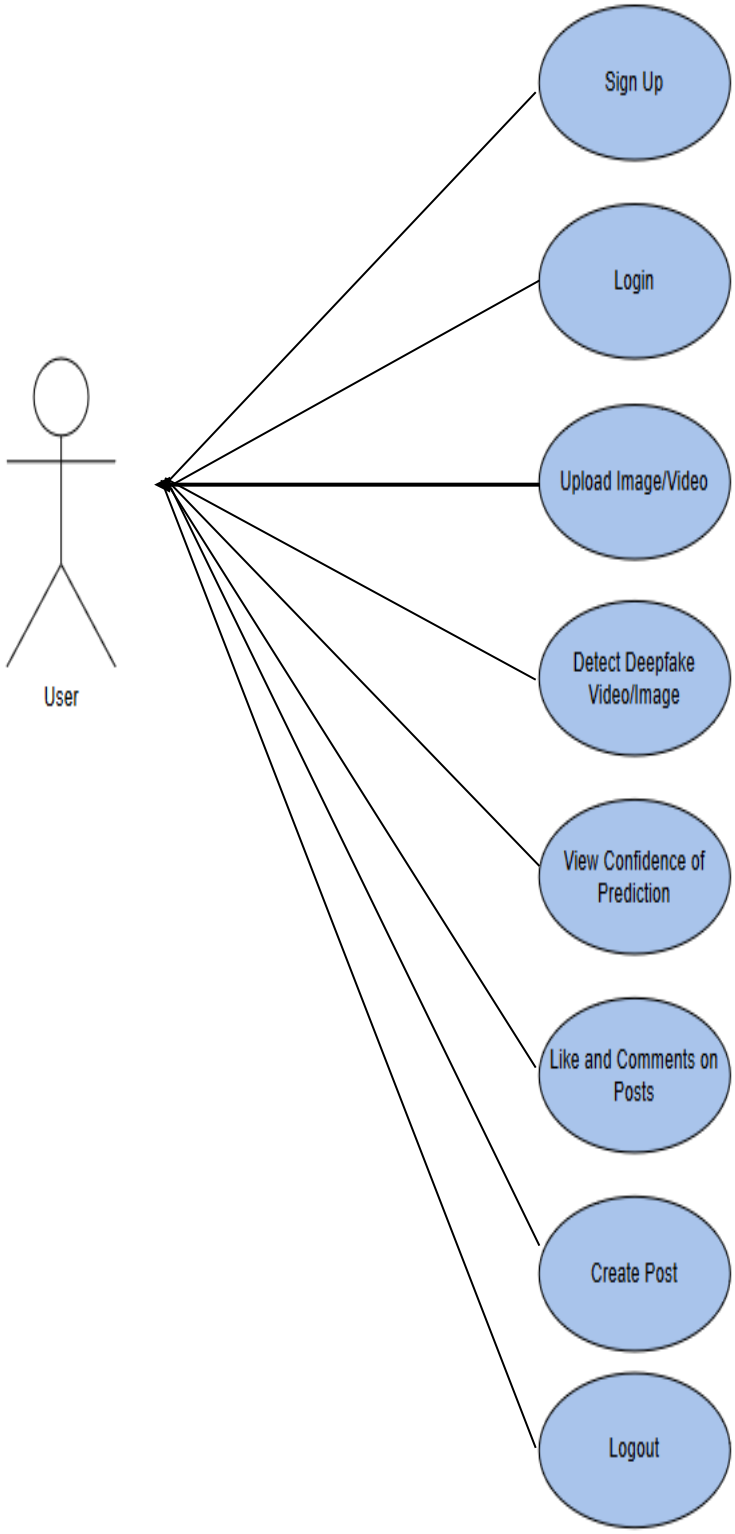


Figure 2: Use Case

3.4.1 Login

Table 4: Login User

Name	Login
Actors	User
Summary	The user shall provide their email and password on the login form and after successful verification, redirect the admin/user to the home page.
Pre-Conditions	The user must register in the system. The user must not already be logged in.
Post-Conditions	The users shall be redirected to the home page of the system and the user's session will be maintained.
Special Requirements	None

Basic Flow			
Actor Action		**System Response**	
1	The user opens the login page.	2	The login page is displayed asking for email and password.
3	The user enters a valid email and password.	4	The system verifies the email and password, establishes a session for the user, and redirects the user to the home page.

Alternative Flow			
3	The user enters an invalid email or password.	4-A	The system responds with an error message: *Incorrect email or password entered.*

3.4.2 Sign Up

Table 5: Sign Up

Name	Signup
Actors	User
Summary	The user shall provide their name, email, password, contact, and address for registration on the signup page and after successful verification, redirect the user to the Main page.
Pre- Conditions	The user should not be already registered in the system.
Post- Conditions	The user shall be able to log in to the system and be redirected to the main page of the system and the user's session will be maintained.
Special Requirements	None

Basic Flow	
Actor Action	**System Response**

1	The user opens the Signup page.	2	The Signup page is displayed asking for name, email, and password.
3	The user enters a valid name, email, and password.	4	The system verifies the name, email, password, and contact to establish a session for the user and redirects the user to the home page.
Alternative Flow			
3	The user enters an invalid name, email, or password.	4-A	The system responds with an error message: *Invalid Credentials.*

3.4.3 Detection

Table 6: Detection

Name	Detection
Actors	User
Summary	The user shall upload a video or Image that will be detected as real or fake.
Pre-Conditions	The user must upload a video or image.
Post-Conditions	The user shall be able to view the result and accuracy score of the fake video

Special Requirements	None

Basic Flow			
Actor Action		**System Response**	
1	The user uploads a video or Image.	2	The system will display con.
3	The user clicks on the upload button.	4	The system checks the video to detect whether it is real or fake content.
Alternative Flow			
3	The user uploads a video that exceeds the length of five minutes.	4-A	The system responds with an error message: *Video too long.*

3.4.4 Community Forum

Table 7: Community Forum

Name	Community Forum
Actors	User
Summary	The user shall be able to communicate with other users.
Pre-Conditions	The user must agree to the community forum's terms of service. The user must be logged in.

Post-Conditions	The users shall be able to detect deepfakes, ask questions, and interact with each other.
Special Requirements	None

Basic Flow			
Actor Action		**System Response**	
1	The user opens the forum to add comments.	2	The community forum is displayed with all comments associated with it.
3	The user clicks reply and post.	4	The system displays a text box for the user to enter a comment and then post it.
Alternative Flow			
3	The user clicks on the “Reply” button and enters a comment that is longer than the maximum length.	4-A	The system responds with an error message: *Comment too long.*

3.4.5 Logout

Table 8: Logout

Name	Logout
Actors	User

Summary	The user shall be able to log out from the system and redirect them to the login page.
Pre- Conditions	The user must log login to the system.
Post- Conditions	The user shall be redirected to the login page of the system.
Special	None

Basic Flow			
Actor Action		**System Response**	
1	The user clicks on the logout button.	2	The system will show a dialog box for confirmation.
3	The user clicks the “Yes” option in the dialog box for logout.	4	The System will remove a session of the user and redirect the user to the login page.
Alternative Flow			
3	The user clicks the “Cancel” option in the dialog box.	4-A	The system will dismiss the dialog box and continue the application flow.

3.5 GUI Graphical User Interfaces

This section should give the GUI dumps of each screen, with reference to the user.

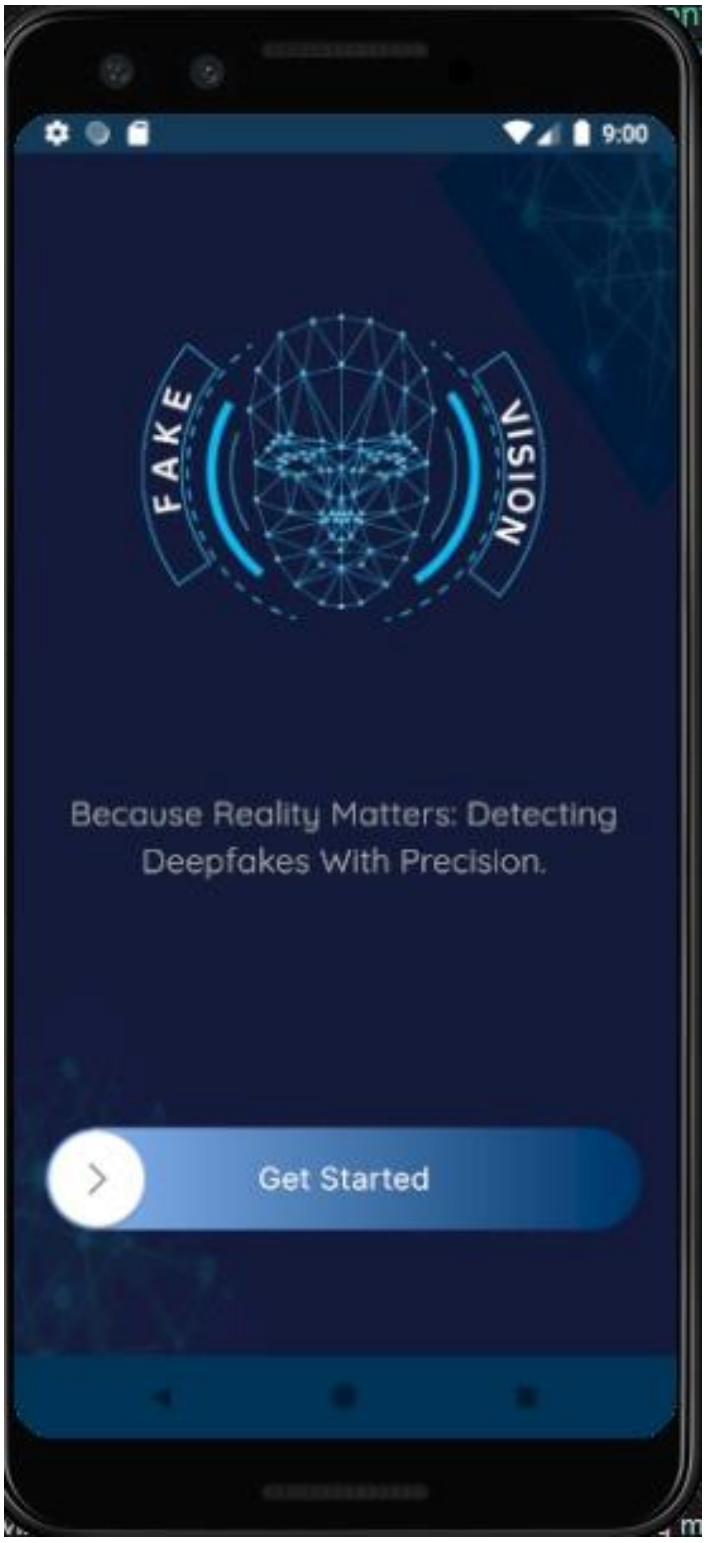


Figure 3: Splash Screen

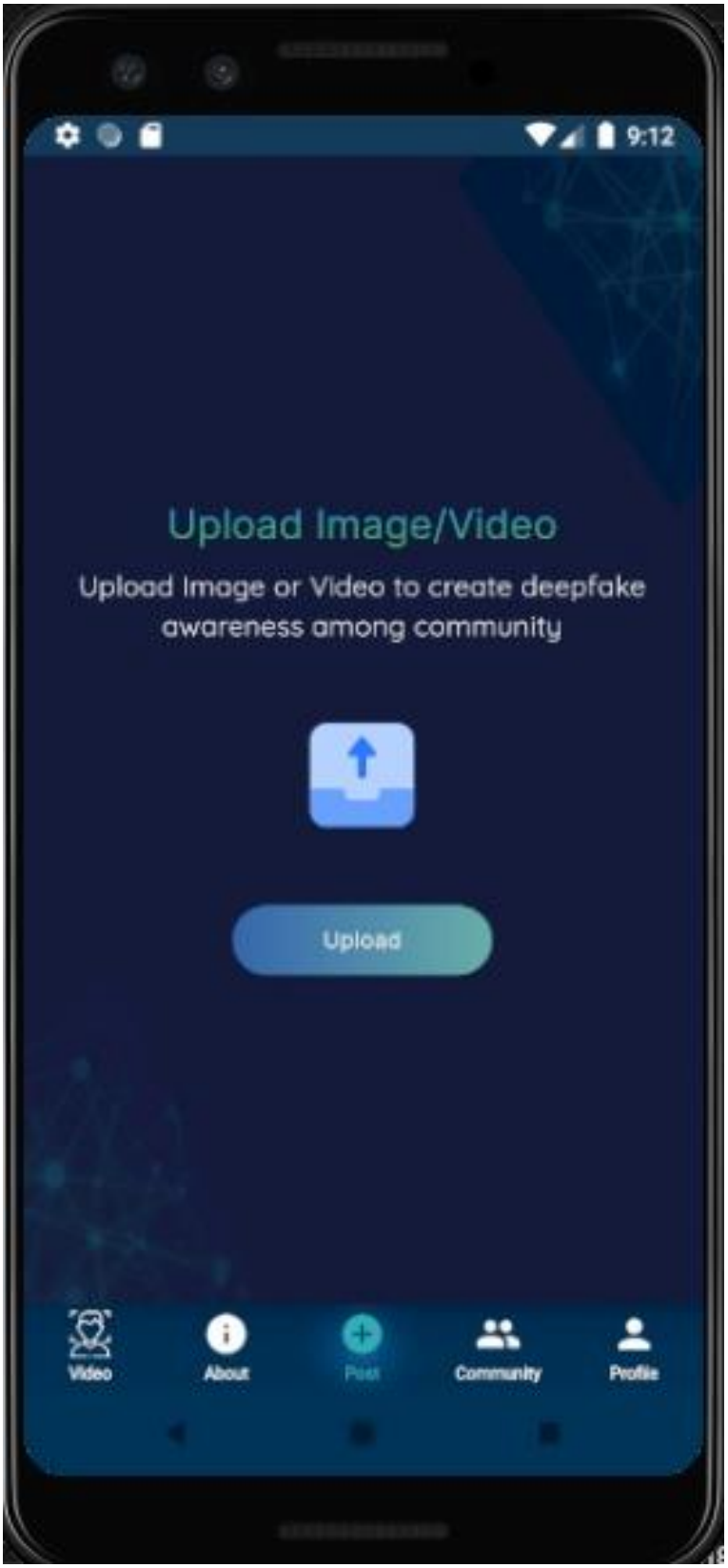


Figure 4: Choose Media

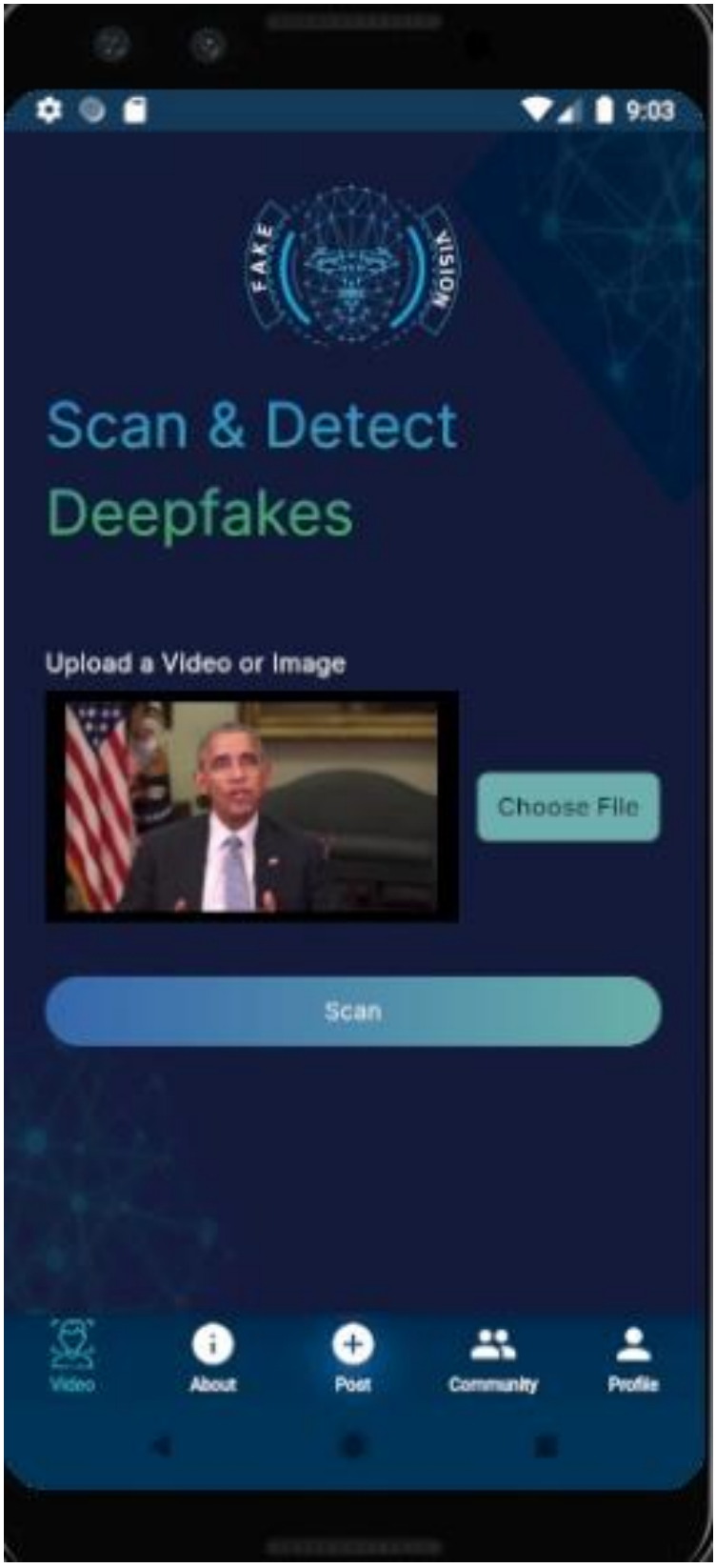


Figure 5: Upload Video/Image

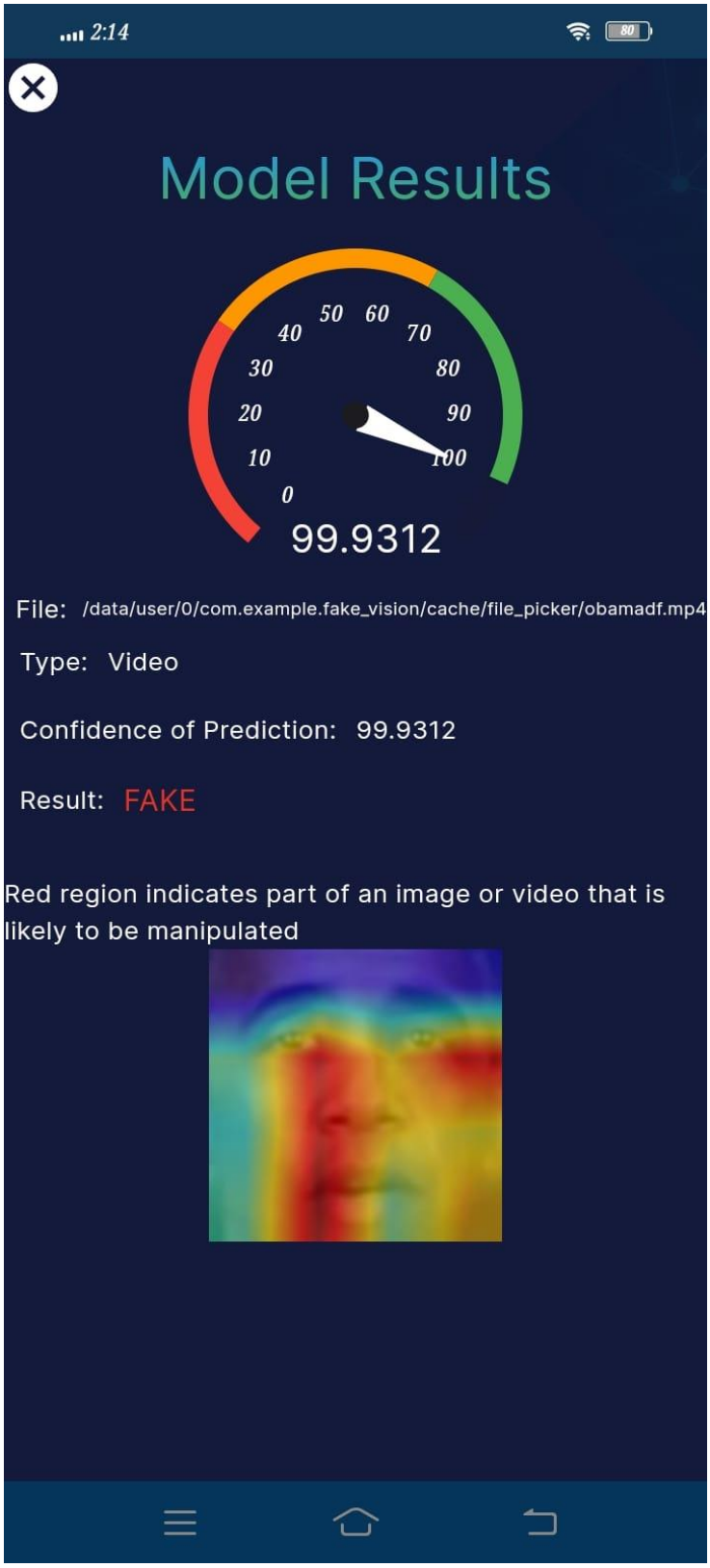


Figure 6: Results



Figure 7: About

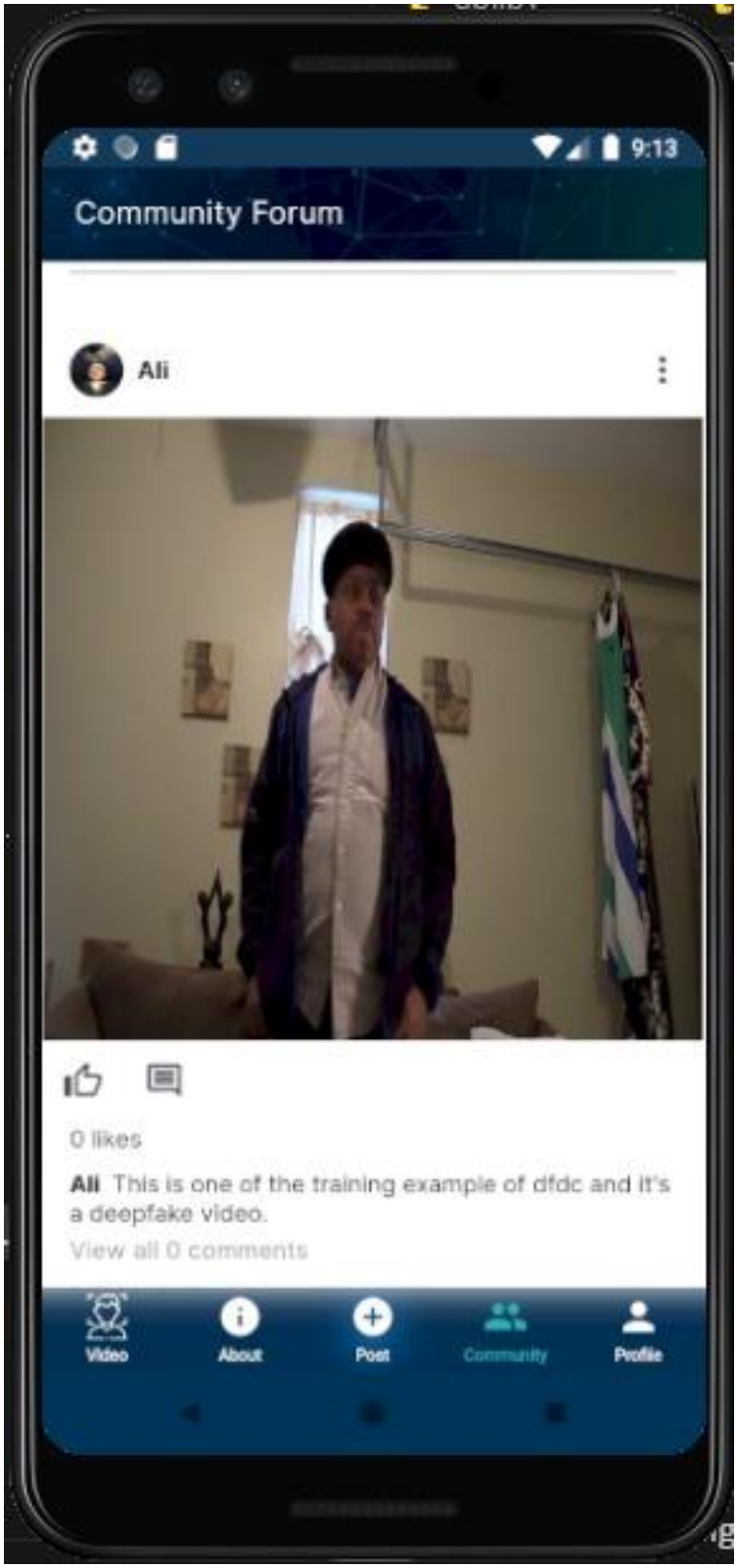


Figure 8: Community Forum

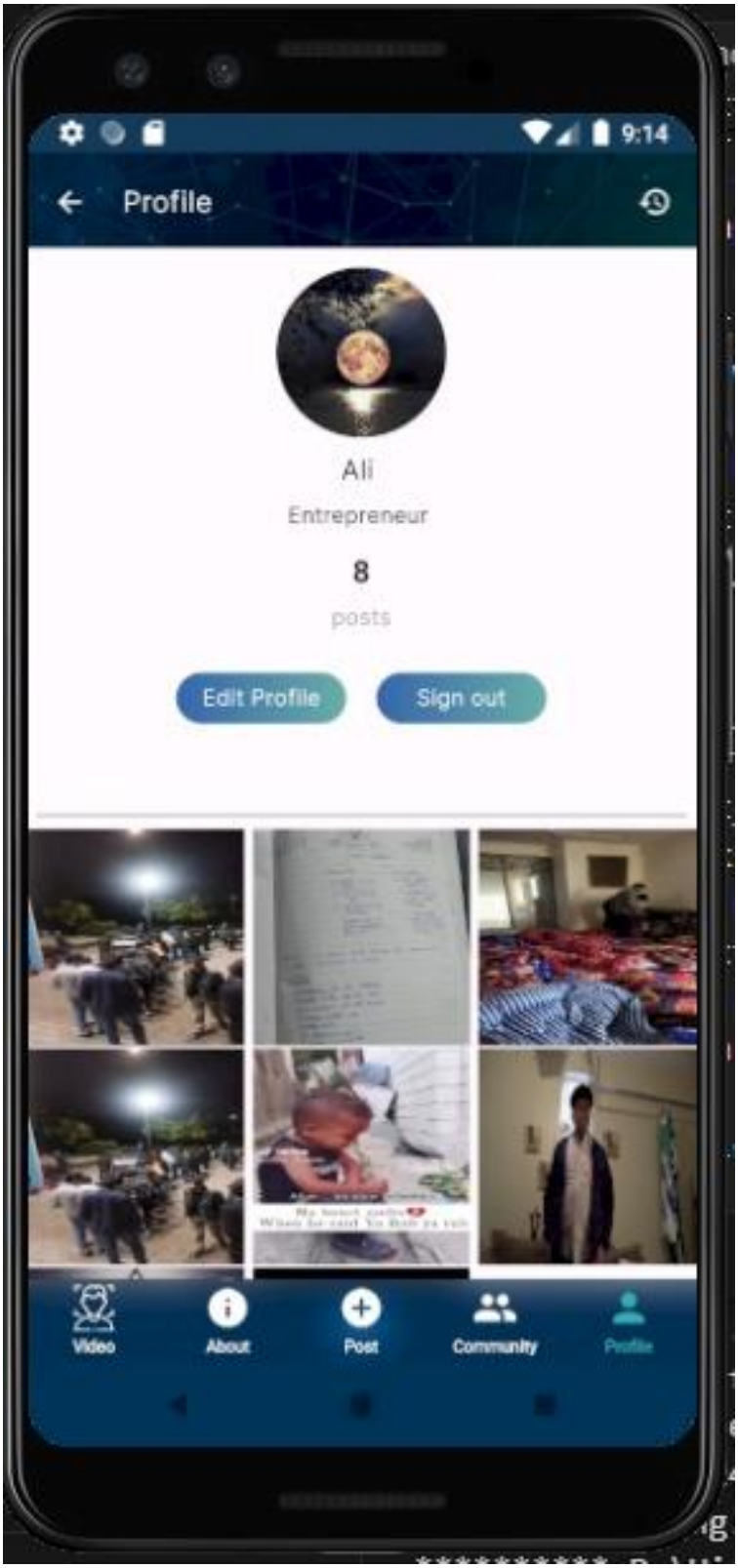


Figure 9: User Profile

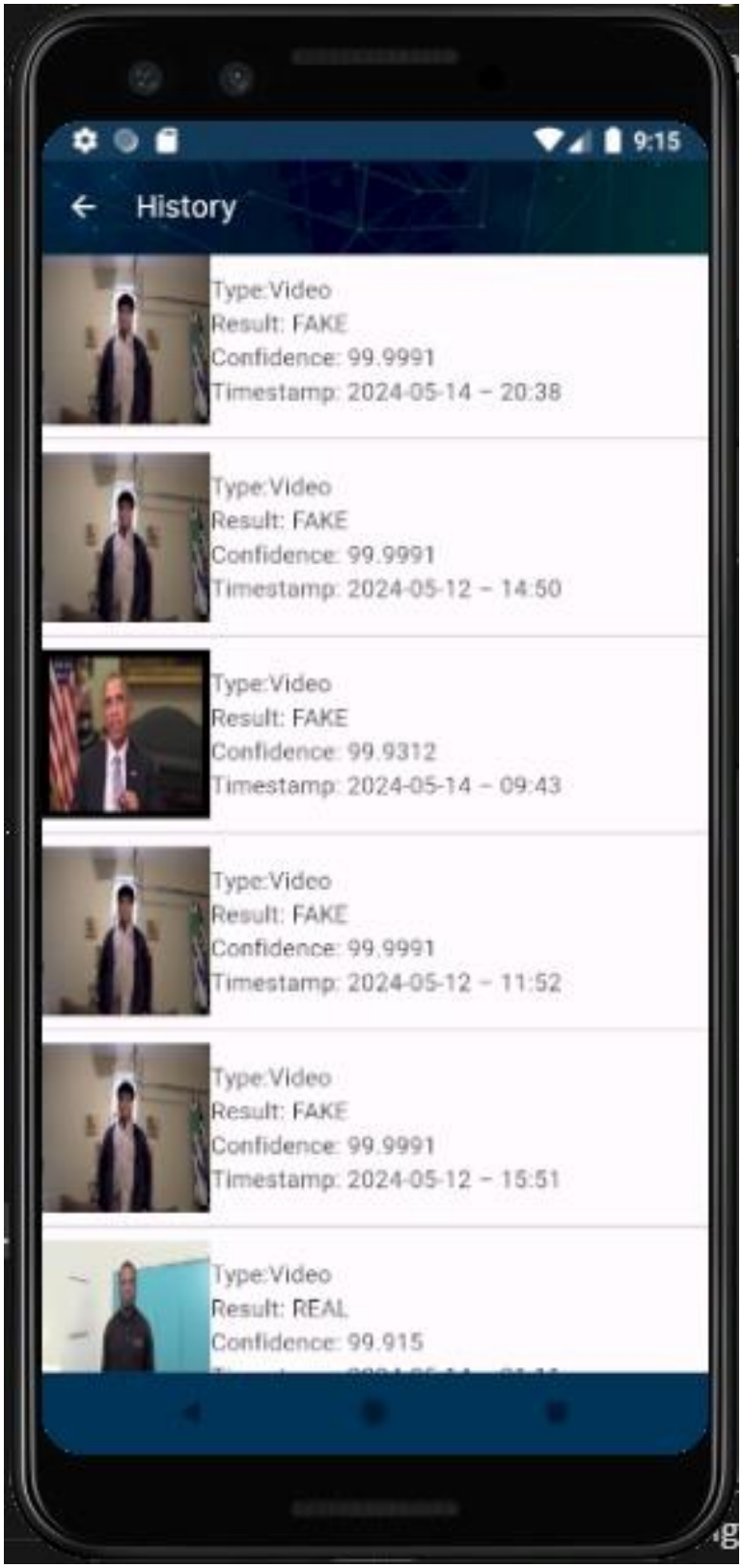


Figure 10: History

Chapter 4: Implementation and Test Cases

4.1 Implementation

App Front-End:

We start the implementation phase of our project by designing our front-end mobile application. By using Flutter and Dart we successfully created the User Interface of our Fake Vision application. The User Interface consists of different modules. It starts by allowing the user to sign up by giving his credentials and he creates his account on our app. We have connected our application with Firebase so users' credentials are saved into our database.

After that user authenticates himself by giving the correct email and password that he has set before, the password and mail will be matched with the email and password in our data and after they are matched successfully, he will be logged in to our app. Then the user comes across the module where he will be given an upload button, by clicking on the upload button he chooses any video that he wants to detect, and then the video will be checked if it's fake or not.

The user will then see the integrity score of his uploaded video. Another module on our application is the community forum as deep fakes is a new technology that emerged after 2015 so people are not well aware of it. On the community forum, we allow users to report and comment on the posts about deep fakes.

Pre-Processing:

Videos were imported using `glob} during the model preparation step, and the mean frame count was computed. For uniformity, a fixed frame count of 150 frames was selected. With {`VideoWriter}, videos were divided into frames, cropped to highlight faces, and then put back together into new videos with a standard resolution of 112 × 112 and 30 frames per second. Every video was rigorously limited to the first 150 frames to preserve temporal sequence integrity for LSTM analysis. By taking these actions, a standardized dataset that is suitable for training deepfake detection models was guaranteed.

Model Training:

In the model training phase, the dataset undergoes a strategic split into training and testing subsets, maintaining a balanced distribution of real and fake videos within each split. With a split ratio of 80% for training (5152 videos) and 20% for testing (1289 videos), the data is efficiently partitioned to ensure robust model evaluation. To facilitate efficient data handling

during training, a data loader is employed, facilitating the loading of videos and their corresponding labels in batches of four.

Training commences with 15 epochs using a learning rate of 1e-5 (0.00001). The adaptive learning rate feature of the Adam optimizer optimizes model parameters, enhancing the convergence of the training process. Employing the Cross-Entropy approach for loss calculation is essential, given the classification nature of the problem at hand. Additionally, a Softmax layer serves as the final layer of the neural network, facilitating multi-class probability determination and yielding confidence scores for predictions. With two output nodes corresponding to "REAL" or "FAKE," the Softmax layer aids in interpreting model outputs with a probabilistic lens.

Following model training, the evaluation process involves the use of a confusion matrix to summarize prediction results and identify classification errors. By quantifying correct and incorrect predictions and segmenting them by class, the confusion matrix provides valuable insights into model performance and areas for improvement. Utilizing this matrix, model accuracy is calculated, offering a comprehensive assessment of its efficacy. Upon successful training and evaluation, the trained model is exported for deployment, enabling real-time predictions on new, unseen data, thereby extending the utility and applicability of the deepfake detection system.

App and Model Integration:

Users can submit video clips for prediction via a link provided by the Flask API, which was developed to integrate the deepfake detection model with the Flutter app. The {/predict_video} API endpoint is capable of receiving POST requests and requires a video file as an argument called `file'. To determine if a video file is real or fraudulent, the API preprocesses it before running it through the deep learning model. The prediction result and confidence level are included in the JSON-formatted response. The confidence level expresses, as a percentage, the degree of assurance in the forecast. By sending a POST request with the video file, users can communicate with the API and they will receive a JSON response with the prediction outcome.

4.1.1 Implementation of First Component/Algorithm

Testing is an essential phase in the development process, crucial for ensuring the quality of the system. Its primary objective is to detect errors and bugs in the system. Software testing

involves evaluating the attributes and functionalities of a software program or system that determines whether the software meets the specified requirements set by the user.

4.2 Test case Design and description

4.2.1 Sample Test Case No.1

Table 9: Test Case 1

User Registration Test			
T-001			
Test Case ID:	*T-001*	**Test Date:**	*Date: 5/16/2024*
Test case Version:	*1*	**Use Case Reference(s):**	*User Registration*
Revision History:	*Refer to previous test case identity (if any)*		
Objective	*User signup/registration*		
Product/Ver/Module:	*Refer to the overall system being built and the place of this test case in it.*		
Environment:	*Signup Page*		
Assumptions:	*The user has a valid email address and access to a device with the internet.*		
Pre-Requisite:	*Internet connection.*		

Step No.	**Execution description**	**Procedure result**
	The user fills out the registration form with valid details and signs up. Check for successful account creation.	*The system processes the registration request and the user account is created successfully.*
Comments: *This test case verifies the functionality of user signup.*		
☒ *Passed* ☐ *Failed* ☐ *Not Executed*		

4.2.1.1 Sample Test Case No.2

Table 10:Test Case 2

Video/Image Detection Test

T-002			
Test Case ID:	*T-002*	**Test Date:**	*Date: 5/16/2024*
Test case Version:	*2*	**Use Case Reference(s):**	*Video/Image Detection*
Revision History:	*Refer to previous test case identity (if any)*		
Objective	*Detection of fake video/image if it is real or fake.*		
Product/Ver/Module:	*Refer to the overall system being built and the place of this test case in it.*		
Environment:	*Detection Page*		
Assumptions:	*The user has a compatible video/image to be detected.*		
Pre-Requisite:	*The user has access to a device with an internet connection.*		

Step No.	Execution description	Procedure result
	Upload a video/image to detect whether it is real or fake.	*The system processes the video/image by applying LSTM and ResNeXt CNN techniques.*

Comments:

This test case verifies the effectiveness of the detection module in identifying fake videos/images.

☒*Passed* ☐*Failed* ☐*Not Executed*

4.2.2 Sample Test Case No.3

Table 11: Test Case 3

File Type Validation Test			
T-003			
Test Case ID:	*T-003*	**Test Date:**	*Date: 5/16/2024*
Test case Version:	*3*	**Use Case Reference(s):**	*Video/Image Detection*
Revision History:	*Refer to previous test case identity (if any)*		
Objective	*Verify system response when word file is uploaded instead of video/image.*		

Product/Ver/Module:	*Refer to the overall system being built and the place of this test case in it.*	
Environment:	*Detection Page*	
Assumptions:	*The user has access to a device with an internet connection.*	
Pre-Requisite:	*The user is on the detection screen.*	
Step No.	**Execution description**	**Procedure result**
	The user uploads a Word file instead of a video/image file.	*System do not allow to choose any other file other then mp4 for video and png and jpg for images.*
Comments: *This test case verifies the system's file type validation for video/image uploads*.		
☒*Passed* ☐*Failed* ☐*Not Executed*		

4.2.3 Sample Test Case No.4

Table 12: Test Case 4

Video/Image File Limit Test			
T-004			
Test Case ID:	*T-004*	**Test Date:**	*Date: 5/16/2024*
Test case Version:	*4*	**Use Case Reference(s):**	*Video/Image Detection*
Revision History:	*Refer to previous test case identity (if any)*		
Objective	*Verify system response when a large video/image file is uploaded exceeding the maximum limit*		
Product/Ver/Module:	*Refer to the overall system being built and the place of this test case in it.*		
Environment:	*Detection Page*		
Assumptions:	*The user has access to a device with an internet connection.*		
Pre-Requisite:	*The user is on the detection screen.*		
Step No.	**Execution description**	**Procedure result**	
	User uploads video or image larger than 17 mbs.	*Connection Reset from Server Side.*	

Comments:
This test case verifies the system's handling of large video/image file uploads.
☒*Passed* ☐*Failed* ☐*Not Executed*

4.2.4 Sample Test Case No.5

Table 13: Test Case 5

Faceless Upload Test			
T-005			
Test Case ID:	*T-005*	**Test Date:**	*Date: 5/16/2024*
Test case Version:	*5*	**Use Case Reference(s):**	*Video/Image Detection*
Revision History:	*Refer to previous test case identity (if any)*		
Objective	*Verify system response when a video/image is uploaded without any detectable faces.*		
Product/Ver/Module:	*Refer to the overall system being built and the place of this test case in it.*		
Environment:	*Detection Page*		
Assumptions:	*The user has access to a device with an internet connection.*		
Pre-Requisite:	*The user is on the detection screen.*		

Step No.	**Execution description**	**Procedure result**
	The user uploads a video/image file without any detectable faces.	*The system will display an error message that states no faces detected, so cannot process the video/image.*
Comments: *This test case verifies the system's ability to detect faces in videos/images uploaded.*		
☒*Passed* ☐*Failed* ☐*Not Executed*		

4.2.5 Sample Test Case No.6

Table 14: Test Case 6

Multiple Faces Video/Image Test			
T-006			
Test Case ID:	*T-006*	**Test Date:**	*Date: 5/16/2024*
Test case Version:	*6*	**Use Case Reference(s):**	*Video/Image Detection*
Revision History:	*Refer to previous test case identity (if any)*		
Objective	*Verify system response when a video/image is uploaded with many detectable faces.*		
Product/Ver/Module:	*Refer to the overall system being built and the place of this test case in it.*		
Environment:	*Detection Page*		
Assumptions:	*The user has access to a device with an internet connection.*		
Pre-Requisite:	*User is on the detection screen.*		
Step No.	**Execution description**	**Procedure result**	
	User uploads a video/image file with many detectable faces.	*System will process the video/image, detect it and give confidence by indicating if the video/image is real or fake.*	
Comments: *This test case verifies the system's performance while handling videos/images with multiple faces.*			
☒*Passed* ☐*Failed* ☐*Not Executed*			

4.2.6 Sample Test Case No.7

Table 15: Test Case 7

Upload Button Validation Test			
T-007			
Test Case ID:	*T-007*	**Test Date:**	*Date: 5/16/2024*
Test case Version:	*7*	**Use Case Reference(s):**	*Video/Image Detection*
Revision History:	*Refer to previous test case identity (if any)*		

Objective	*Verify system response when attempting to upload without selecting a video/image.*	
Product/Ver/Module:	*Refer to the overall system being built and the place of this test case in it.*	
Environment:	*Detection Page*	
Assumptions:	*User has access to device with internet connection.*	
Pre-Requisite:	*User is on the detection screen.*	
Step No.	**Execution description**	**Procedure result**
	User presses the upload button without selecting any video/image	*System will display an error message: "Please select a video/image."*
Comments: *This test case verifies the system's validation for video/image selection before uploading.*		
☒*Passed* ☐*Failed* ☐*Not Executed*		

4.2.7 Sample Test Case No.8

Table 16: Test Case 8

Real Video/Image Detection Test			
T-008			
Test Case ID:	*T-008*	**Test Date:**	*Date: 5/16/2024*
Test case Version:	*8*	**Use Case Reference(s):**	*Video/Image Detection*
Revision History:	*Refer to previous test case identity (if any)*		
Objective	*Verify system response when a real (not fake) video/image is uploaded*		
Product/Ver/Module:	*Refer to the overall system being built and the place of this test case in it.*		
Environment:	*Detection Page*		
Assumptions:	*User has access to device with internet connection.*		
Pre-Requisite:	*User is on the detection screen.*		
Step No.	**Execution description**		**Procedure result**

	User uploads a real video/image.	*The system processes the video/image and provides confidence of prediction that the image/video is real.*
Comments: *This test case verifies the system's accuracy in detecting the real video/image.*		
☒*Passed* ☐*Failed* ☐*Not Executed*		

4.2.8 Sample Test Case No.9

Table 17: Test Case 9

Cropped Faces Detection Test			
T-009			
Test Case ID:	*T-009*	**Test Date:**	*Date: 5/16/2024*
Test case Version:	*9*	**Use Case Reference(s):**	*Community Forum*
Revision History:	*Refer to previous test case identity (if any)*		
Objective	*Verify the system's response when a video/image is uploaded with cropped faces.*		
Product/Ver/Module:	*Refer to the overall system being built and the place of this test case in it.*		
Environment:	*Detection Page*		
Assumptions:	*Users can interact with other users and like or comment on posts.*		
Pre-Requisite:	*The user is on the detection screen.*		
Step No.	**Execution description**		**Procedure result**
	The user uploads a video/image with cropped faces		*The system processes the video/image and will show a percentage score indicating if the video is fake or not.*
Comments: *This test case verifies the system's handling of videos/images with partially cropped faces.*			
☒*Passed* ☐*Failed* ☐*Not Executed*			

4.2.9 Sample Test Case No.10

Table 18: Test Case 10

Confidence of prediction			
T-010			
Test Case ID:	*T-010*	**Test Date:**	*Date: 5/16/2024*
Test case Version:	*10*	**Use Case Reference(s):**	*Confidence of Prediction*
Revision History:	*Refer to previous test case identity (if any)*		
Objective	*How much confident system is that a video/image is real or fake.*		
Product/Ver/Module:	*Refer to the overall system being built and the place of this test case in it.*		
Environment:	*Detection Page*		
Assumptions:	*The user has a compatible video/image to be detected.*		
Pre-Requisite:	*The user has access to a device with an internet connection.*		
Step No.	**Execution description**	**Procedure result**	
	The user uploads video/image to check if it is real or fake.	*The system will generate confidence of prediction and predict how much real or fake the media is.*	
Comments: *This test case verifies the accuracy score of a video/image.*			
☒*Passed* ☐*Failed* ☐*Not Executed*			

4.3 Test Metrics

Summarize here the common ground of attributes of test case metrics.

4.3.1 Sample Test case Matric.No.1

Table 19: Test Metrices

Metric:	**Purpose**
Number of Test Cases:	10

Number of Test Cases Passed:	10
Number of Test Cases Failed:	0
Test Case Defect Density:	(No of test cases failed * 100) No test cases executed
Test Case Effectiveness:	No defects detected using test cases *100 Total number of defects detected
Traceability Matrix:	Traceability is the ability to determine that each feature has a source in requirements and each requirement has a corresponding implemented feature.

Chapter 5: Experimental Results and Analysis

5.1 Confidence of Prediction:

The confidence of prediction function was evaluated by comparing the result generated by the app with the test media. To evaluate the efficacy of the detection function, a variety of test deepfake images and videos produced by artificial intelligence were uploaded into the application. After that, the app's output was compared to see whether or not it could correctly identify them.

5.2 User Engagement Analysis:

Surveys of users' opinions and usability testing were done to evaluate the application's overall user experience. The tasks assigned to users included uploading photos or videos to identify deep fakes, searching for other users, sharing and uploading media in the community forum, and leaving comments on posts. We collected and examined their comments and observations. Users found the community forum entertaining, the detection feature accurate, and the interface easy to use indicating a positive overall user experience.

5.3 Bug Reporting and Troubleshooting:

Bugs and errors were discovered and reported throughout the testing stage. The development team immediately addressed these issues and fixed bugs in subsequent releases. The team's evaluation of bug reports and issue remedies showed their dedication to producing a trustworthy and error-free product.

5.4 Vulnerability Analysis:

The application has undergone a rigorous security assessment to identify any potential flaws or threats. Vulnerability scanning was performed to determine the efficacy of the application's security mechanisms. After analysis, it was determined that the program has put in place the right security measures such as data encryption, and strong user authentication mechanisms to protect user data and maintain data privacy.

5.5 App Capability Analysis:

To ensure compatibility, the program was tested across several devices, operating systems, and screen sizes. The evaluation validated that the application ran well across several platforms while providing a seamless user interface. Any compatibility concerns that arose were addressed and fixed throughout the testing phase.

5.6 Conclusion:

In summary, the trial data demonstrated that the Fake Vision Mobile Application performed well, providing precise data, a positive user experience, and adhering to security criteria. It demonstrated stability, quick response, and compatibility across a variety of devices and platforms. The extensive testing, training, and analysis performed during the development period ensured the application's dependability and functioning, resulting in a high-quality offering for users.

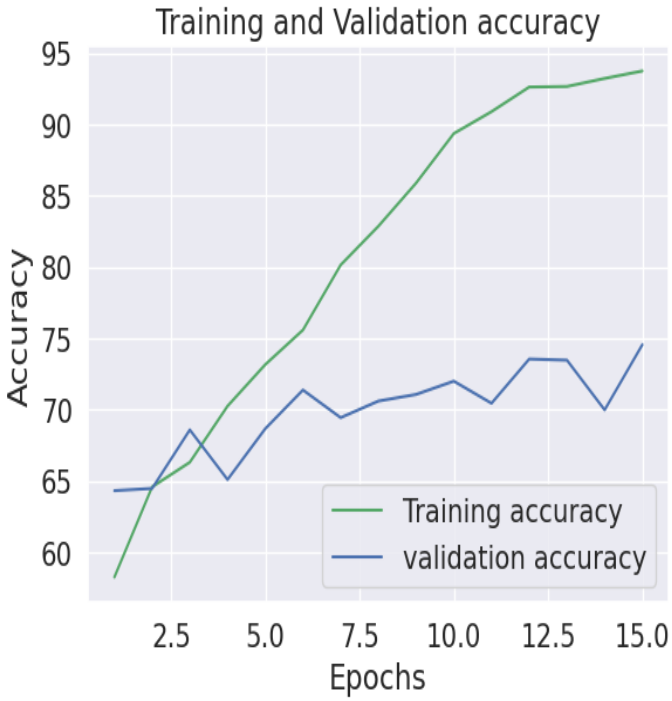


Figure 11: Accuracy Curve

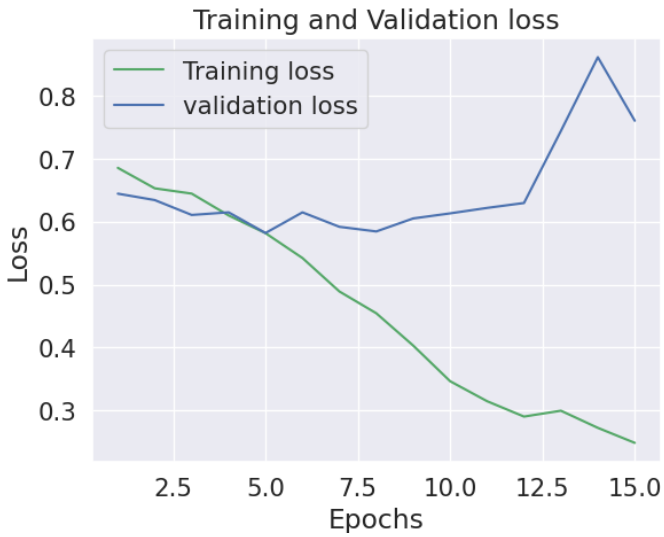


Figure 12: Loss Curve

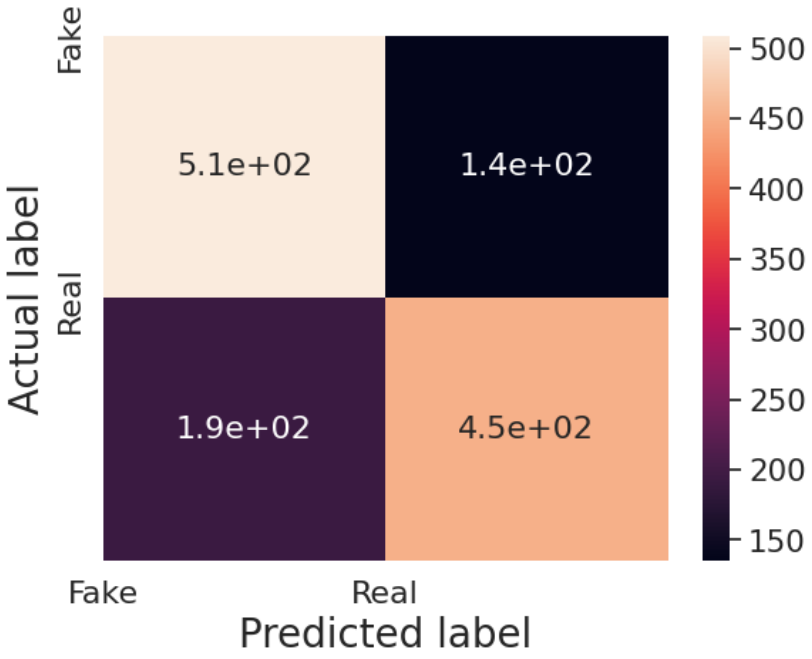


Figure 13: Confusion Matrix

The above figures shows that the model achieves a training accuracy of 93%, indicating strong performance on the training data, but its test accuracy drops to 73%. The loss curve shows the rate at which the model predicts real media as fake and fake media as real. The greater the loss is the lower will be the accuracy.

Chapter 6: Conclusion and Future Directions

As our project comes to a close and we move forward, we have gained valuable experience and achieved significant milestones in developing the front end of our Fake Vision mobile application, resolving UI implementation issues, and working effectively as a team. Our focus has been on training our model to identify deepfake videos created through face-swapping using PyTorch, Python, and neural networks such as LSTM and RNN.

Our goal is to tackle the persistent problem of deepfake technology while enhancing and refining our application. We have demonstrated confidence in the proposed model and a neural network-based approach to determine whether a video is a deepfake or real. By analyzing one second of video at a rate of ten frames per second, our technique can accurately predict the outcome. To implement the model, we utilized an LSTM for temporal sequence processing and a pre-trained ResNeXt CNN model to extract frame-level features.

6.1 Introduction:

We would like to include the lessons, achievements, and results that we have learned while developing this project. What challenges we have faced and how we have overcome these challenges by first doing our research, requirement gathering, implementation, and solution, we would like to add the scope of our project, which domain it covers successfully, and what is left. At the end, we will highlight our optimal solution to the problem which fits well as well as further improvements which we would like to add in the future.

6.2 Achievements and Future Work:

We have completed the front-end development of our Fake Vision Mobile Application by implementing the app's UI. For the app's functionality, we have integrated it with Firebase. We utilized the FakeCeleb DF, FaceForensics++, and DFDC datasets. Using Deep Learning algorithms such as RestNeXt and LSTM, we trained a model to detect deepfake videos and images. We then integrated this model into our app to offer an optimal solution for identifying deepfakes, which we call **"FAKE VISION."**

6.3 Future Work:

There is always room for improvement in any developed system, especially when the project is built using the latest trending technology and has a promising future. Currently, we have

developed an app to detect deepfakes using faces. In the future, the research could be extended to detect full-body deepfakes and to determine whether the audio in the video is authentic or not.

References

[1] D. a. E. J. D. Güera, “Deepfake video detection using recurrent neural networks,” IEEE, 2018..

[2] R. C. a. E. R. M. Staudemeyer, “Understanding LSTM--a tutorial into long short-term memory recurrent neural networks,” 2019.

[3] FaceForensics++. [Online]. Available: https://github.com/ondyari/FaceForensics.

[4] Kaggle DFDC , [Online]. Available: https://www.kaggle.com/c/deepfake-detection-challenge/data.

[5] C. Deepfake. [Online]. Available: https://github.com/yuezunli/celeb-deepfakeforensics.

[6] Deepware Scanner, 2021. [Online]. Available: https://scanner.deepware.ai/.